

ESO-207: Theoretical Assignment-3

Aviral Gupta
230246

Indian Institute of Technology, Kanpur

Question 1: Restoring Galactic Communication Network

The **Galactic Federation** maintains a communication network consisting of n planets, numbered from 1 to n , connected by m bidirectional communication channels. After a severe solar storm, parts of the network have been damaged, leaving several planets disconnected.

To restore communication, engineers can construct new channels between any two planets i and j . However, the construction cost is proportional to the square of the difference in their identification numbers:

$$\text{Cost}(i, j) = (i - j)^2$$

The goal is to ensure that **Planet 1** (the command center) and **Planet n** (the outer colony) are connected—either directly or indirectly—by adding at most two new channels. The objective is to determine the minimum total cost required to achieve this connectivity.

Question (a)

Design an efficient algorithm to compute the minimum total cost needed to make Planet 1 and Planet n connected by adding at most two new edges. Clearly describe the main steps of your approach and justify any data structures used.

Solution:

Idea:

We are required to find the minimum total cost required to make Planet 1 and Planet n connected. The key observation is that the graph may consist of several disconnected components. If Planets 1 and n are already in the same component, the cost is 0. Otherwise, we may connect them either directly (by one new edge) or indirectly via another component (using two new edges). Since the cost depends only on the difference in planet indices, we can compute the closest possible pairs efficiently using index-based distances rather than comparing all pairs explicitly.

Main Steps:

1. **Identify connected components:** Perform a Breadth-First Search (BFS) over the graph to find all connected components. Each node is assigned a component ID. If Planet 1 and Planet n belong to the same component, the answer is immediately 0.
2. **Compute index-based distances:** Let C_1 be the component containing Planet 1, and C_n the component containing Planet n . For every planet i , compute:

$$d_{C_1}[i] = \text{minimum } |i - j| \text{ where } j \in C_1$$

$$d_{C_n}[i] = \text{minimum } |i - j| \text{ where } j \in C_n$$

These distances are efficiently computed in $\mathcal{O}(n)$ time using a two linear pass.

3. **Evaluate connection costs through each component:** For every component C , determine:

$$\text{min1} = \min_{v \in C} (d_{C_1}[v])^2, \quad \text{minn} = \min_{v \in C} (d_{C_n}[v])^2$$

If both values are finite, connecting $C_1 \rightarrow C \rightarrow C_n$ requires total cost:

$$\text{Cost} = \text{min1} + \text{minn}$$

The minimum of these over all components gives the optimal cost to connect Planets 1 and n with at most two new edges.

4. **Output the minimum total cost:** Finally, print the smallest computed cost.

Justification of Data Structures:

- **Adjacency List** ($\text{adj}[1..n]$) Used to represent the communication network between planets. It enables efficient graph traversal algorithms such as BFS and DFS, allowing operations like connectivity checks and shortest-path exploration in $\mathcal{O}(n + m)$ time.
- **Component Array** ($\text{comp}[1..n]$) Stores the component identifier of each planet, allowing constant-time checks of which component a planet belongs to. Essential for quickly determining whether Planets 1 and n are already connected.
- **Component List** ($\text{components}[1..\text{compCount}]$) Each entry stores the list of planets belonging to a specific component. This structure is crucial for iterating through each component when calculating potential connection costs.
- **Distance Arrays** ($\text{dc1}[1..n]$, $\text{dcn}[1..n]$) Store the minimum index distance of each planet from the component containing Planet 1 and the component containing Planet n , respectively. These are computed efficiently using two linear sweeps over all planets.
- **Boolean Marker Arrays** ($\text{isC1}[1..n]$, $\text{isCn}[1..n]$) Used to mark planets that belong to the components of Planet 1 and Planet n . These arrays guide the distance computation process without repeatedly searching through components.

Question (b)

Discuss why the algorithm guarantees the minimum possible cost and provide pseudo code of your approach from part (a).

Solution:

Proof of Correctness:

The algorithm guarantees the minimum possible cost because it systematically considers all valid ways to connect Planet 1 and Planet n using at most two new channels:

- Each connected component is treated as an independent cluster of already-connected planets.
- The algorithm first checks whether Planets 1 and n are already connected, in which case the minimum cost is trivially 0.
- For disconnected cases, it computes for every planet i the minimum index distance to any planet in the component of Planet 1 (C_1) and to any planet in the component of Planet n (C_n). These distances ensure that for any potential new edge (i, j) , the cost $(i - j)^2$ is minimized since $|i - j|$ is the smallest possible index difference.
- When evaluating through an intermediate component C , the algorithm computes:

$$\text{Cost}(C) = \min_{v \in C} (d_{C_1}[v])^2 + \min_{v \in C} (d_{C_n}[v])^2$$

This expression captures the minimal combined cost of adding one new channel from C_1 to C and another from C to C_n . The minimum of these costs over all components yields the optimal total cost to connect Planet 1 and Planet n .

- Because all possible single or double edge additions are implicitly considered through distance computations, no cheaper connection can exist that the algorithm overlooks.

Thus, the algorithm always finds the minimum possible cost to connect the two planets, either directly or through one intermediate component.

Pseudocode:

Algorithm 1: MINCONNECTCOST($n, m, edges$)

Input: Number of planets n , number of existing channels m , list of edges (u, v)

Output: Minimum total cost to connect Planet 1 and Planet n

```

1 Step 1: Identify connected components
2 Build adjacency list  $adj[1..n]$  from the given edges
3 Initialize array  $comp[1..n] \leftarrow -1$ 
4  $compCount \leftarrow 0$ 
5 for  $i \leftarrow 1$  to  $n$  do
6   if  $comp[i] = -1$  then
7      $compCount \leftarrow compCount + 1$ 
8     perform BFS from  $i$  assigning  $comp[v] \leftarrow compCount$  for all reachable  $v$ 
9     store all such  $v$  in list  $components[compCount]$ 
10 if  $comp[1] = comp[n]$  then
11   return 0
12 Step 2: Compute index-based distances
13 Let  $C_1 \leftarrow$  component of Planet 1,  $C_n \leftarrow$  component of Planet  $n$ 
14 Initialize arrays  $dc1[1..n] \leftarrow \infty$ ,  $dcn[1..n] \leftarrow \infty$ 
15 For all  $v \in C_1$ , set  $dc1[v] \leftarrow 0$ 
16 For all  $v \in C_n$ , set  $dcn[v] \leftarrow 0$ 
17 Perform two linear sweeps (left-to-right and right-to-left) to compute:
18    $dc1[i] =$  minimum distance in index to any  $j \in C_1$ 
19    $dcn[i] =$  minimum distance in index to any  $j \in C_n$ 
20 Step 3: Evaluate cost through each component
21  $best \leftarrow \infty$ 
22 for each component  $C$  do
23    $min1 \leftarrow \infty$ ,  $minn \leftarrow \infty$ 
24   for each planet  $v \in C$  do
25      $min1 \leftarrow \min(min1, dc1[v]^2)$ 
26      $minn \leftarrow \min(minn, dcn[v]^2)$ 
27   if  $min1 < \infty$  and  $minn < \infty$  then
28      $best \leftarrow \min(best, min1 + minn)$ 
29 Step 4: Output result
30 return  $best$ 

```

Question (c)

Analyze the time and space complexity of your code from part (b).

Solution:

Time Complexity Analysis:

- **Building the adjacency list:** Each of the m bidirectional channels is processed once and added to two adjacency lists, giving a cost of $\mathcal{O}(m)$.
- **Finding connected components (BFS traversal):** Each planet and each edge is visited exactly once during the BFS process. Therefore, the total cost for this step is $\mathcal{O}(n + m)$.

- **Computing index-based distances:** The two linear sweeps (left-to-right and right-to-left) across all planets each take $\mathcal{O}(n)$ time. Hence, this step costs $\mathcal{O}(n)$ in total.
- **Evaluating minimum connection cost:** Each planet belongs to exactly one component, and all components are processed once. For each planet, we check its precomputed distances in constant time, giving another $\mathcal{O}(n)$ step.
- **Final comparison and output:** Selecting the minimum cost among all components is done in $\mathcal{O}(1)$ per component, thus $\mathcal{O}(n)$ overall.

Combining all parts, the total time complexity is:

$$\mathcal{O}(n + m)$$

This is optimal since every node and edge must be examined at least once to determine connectivity.

Space Complexity Analysis:

- The adjacency list stores each edge twice, requiring $\mathcal{O}(n + m)$ space.
- Arrays such as `comp`, `dc1`, `dcn`, `isC1`, and `isCn` each use $\mathcal{O}(n)$ space.
- The `components` structure stores all nodes grouped by component, taking $\mathcal{O}(n)$ total.
- The BFS queue uses at most $\mathcal{O}(n)$ additional space in the worst case.

Therefore, the overall space complexity is:

$$\mathcal{O}(n + m)$$

Question 2: The Postal Routes of Valori

In the ancient kingdom of **Valoria**, there are n towns connected by m two-way trade routes. Each route connects two distinct towns and can be traveled in both directions.

The King wishes to establish a grand postal route for royal messengers. The rules for the route are as follows:

- Every trade route must be used exactly once during the journey.
- A messenger may travel along the routes in any direction, but cannot reuse any route.
- The postal routes starts and ends in different towns.

Question (a)

Design an efficient algorithm to decide whether a valid royal postal route exists in the kingdom of Valoria. If it exists, also provide such a route. Explain your approach.

Solution:

Idea and Explanation:

The goal is to find a continuous route that uses every trade route exactly once, starting and ending in different towns. For this to be possible, the structure of the network must allow smooth movement through all connections without leaving any route unused.

The feasibility of such a route depends mainly on how many trade routes meet at each town (its degree) and whether all towns are connected together.

- Whenever the messenger enters a town by one route, they must leave through another unused route to continue the journey.
- Thus, all towns in between the path must have an even number of connecting routes so that entries and exits pair up.
- Only two towns can have an odd number of connecting routes, start and end point.
- If more than two towns have an odd number of routes then it will be no possible.
- If all towns have even routes, the journey would start and end at the same town, which is not allowed in this problem.
- When these conditions are satisfied, starting from one of the towns with an odd number of routes and following unused routes one by one ensures that every route is covered exactly once and the journey ends at the other odd-degree town.

Pseudocode:**Algorithm 2:** ROYALPOSTALROUTE($n, m, edges$)

Input: Number of towns n , routes m , list of routes (u, v)
Output: A valid postal route or -1 if none exists

```

1 Initialize: adjacency list  $adj[1..n]$ , degree array  $deg[1..n] \leftarrow 0$ ,  $used[1..m] \leftarrow false$ 
2 for  $i \leftarrow 1$  to  $m$  do
3    $read(u, v)$ 
4    $add(v, i)$  to  $adj[u]$ ,  $(u, i)$  to  $adj[v]$ 
5    $deg[u] \leftarrow deg[u] + 1$ ,  $deg[v] \leftarrow deg[v] + 1$ 
6  $odd \leftarrow$  list of towns with odd  $deg$ 
7 if  $|odd| \neq 2$  then
8   return -1
9  $start \leftarrow$  one town from  $odd$ 
10 if network not connected from  $start$  then
11   return -1
12 Initialize: stack  $S \leftarrow [start]$ , list  $path$ 
13 while  $S$  not empty do
14    $u \leftarrow$  top of  $S$ 
15   if  $adj[u]$  not empty then
16      $(v, id) \leftarrow$  last element of  $adj[u]$ 
17     remove it from  $adj[u]$ 
18     if not  $used[id]$  then
19        $used[id] \leftarrow true$ 
20       push  $v$  onto  $S$ 
21   else
22     append  $u$  to  $path$ 
23     pop  $S$ 
24 return  $path$  (reversed)

```

Question (b)

The royal messengers have now requested you to find a route that starts and ends in the same town. Mention the changes you need to make in your algorithm in part (a).

Solution:**Idea and Explanation for the Modified Route:**

Now, the royal messengers wish to begin and end their journey in the same town. This changes the condition on how many towns can have an odd number of connecting routes. In this case, every time the messenger enters a town, they must also leave through another unused route — including the starting town — because the journey forms a complete loop.

- Since the route starts and ends at the same town, no town should have an unmatched entry or exit.
- Therefore, **every town must have an even number of connecting routes** so that each entry has a corresponding exit.
- The entire trade network must still be connected so that all routes can be traversed without leaving any unused.
- In the algorithm, instead of checking for exactly two towns with an odd number of routes, we now check that **no town has an odd number of routes**.
- The rest of the algorithm — connectivity check and route construction — remains the same, but we can start the journey from any town.

- This ensures that the messenger completes a single continuous loop, using every trade route exactly once and returning to the starting town.

Question (c)

Analyze the time and space complexity of your algorithm for both the parts (a) and (b).

Solution:

Time and Space Complexity Analysis:**Part (a): Route Starting and Ending in Different Towns**

- Building the adjacency list requires traversing all m routes once, which takes $O(m)$ time.
- Counting the number of connecting routes (degrees) for each of the n towns takes $O(n)$ time.
- The connectivity check using BFS or DFS visits each town and route once, contributing $O(n + m)$ time.
- Constructing the route by following and marking each trade route exactly once also takes $O(m)$ time.
- Therefore, the overall time complexity is:
$$O(n + m)$$
- The adjacency list stores all m routes (two entries per route) and additional arrays for degrees, visited status, and used routes, leading to:

$O(n + m)$ space complexity.

Part (b): Route Starting and Ending in the Same Town

- The modified algorithm only changes the degree condition (now all towns must have even degrees) but follows the same steps: adjacency construction, connectivity check, and route generation.
- Thus, the operations remain the same asymptotically.
- Hence, both the time and space complexities are identical to part (a):

Time Complexity: $O(n + m)$, Space Complexity: $O(n + m)$

Question 3: The Sky Trams of Aetheria

In the floating city of **Aetheria**, hundreds of sky trams glide endlessly between levitating islands. Each tram follows a fixed circular route, repeating the same sequence of islands forever. There are n tram routes, where the i^{th} route is represented by a list of atmost m distinct island identifiers that the tram visits in order. For example, if `route[0] = [1, 2, 3, 4]`, it means that the first tram travels as

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow \dots$$

A messenger begins their journey at island `source` and wishes to reach island `target`. The messenger may board or exit a tram only at an island that the tram visits. Two trams are said to **intersect** if they both stop at the same island, allowing the messenger to switch from one tram to another.

Your task is to determine the smallest number of trams the messenger must use to travel from `source` to `target`. If it is impossible to reach the target island, report that as well.

Question (a)

Formulate the problem using a graph. Explain your formulation and mention what do the nodes and edges represent in your graph.

Solution:

Graph Formulation and Explanation:

We can model the network of tram routes as a graph to determine the minimum number of trams required to travel from a given source island to a target island.

- Each **node** in the graph represents a **tram route**.
- There is an **edge** between two nodes (routes) if the corresponding tram routes share at least one common island. This indicates that a messenger can switch trams at that island.
- Additionally, we introduce two **extra nodes** to represent the **source** and **target** islands.
- Connect the **source node** to all tram routes that stop at the source island.
- Connect all tram routes that stop at the target island to the **target node**.
- The problem now reduces to finding the **shortest path** from the source node to the target node in this graph. Each edge traversed corresponds to using one tram.
- If no path exists between the source and target nodes, it is impossible for the messenger to reach the target island.

Question (b)

Design an efficient algorithm to compute the minimum number of trams required to travel from source to target. Describe your approach.

Solution:

Idea and Explanation:

To compute the minimum number of trams required to travel from a source island to a target island, we first model the problem as a graph. Once the graph is built, the problem reduces to finding the shortest path from the source node to the target node, where each edge corresponds to using one tram.

Using Breadth-First Search (BFS) ensures that we find the minimum number of trams efficiently, as BFS explores all possible sequences of tram transfers in order of increasing tram count.

Algorithm:

- Build a mapping from each island to the list of tram routes that stop there.
- Construct nodes for each tram route and add two extra nodes for the source and target islands.
- Connect the source node to all tram routes containing the source island, and connect all tram routes containing the target island to the target node.
- Perform BFS starting from the source node, keeping track of the number of trams used to reach each node.
- Whenever a node corresponding to the target is reached, return the current tram count as the minimum number of trams.
- If BFS completes without reaching the target node, report that the target island is unreachable.

Question (c)

Provide a pseudocode to implement your approach

Solution:**Pseudocode:****Algorithm 3:** MINTRAMS(*routes*, *source*, *target*)

Input: List of tram routes, source island, target island

Output: Minimum number of trams required, or -1 if unreachable

1 Step 1: Build mappings and graph

- 2 Map each island to the list of tram routes that stop there
- 3 Create nodes for each tram route
- 4 Add two extra nodes: *s* (source) and *t* (target)
- 5 Connect *s* to all routes containing the source island
- 6 Connect all routes containing the target island to *t*

7 Step 2: Initialize BFS

- 8 Initialize a queue *Q* and enqueue *s* with tram count 0
- 9 Mark *s* as visited

10 Step 3: BFS traversal

- 11 **while** *Q* not empty **do**
- 12 Dequeue (*u*, *cnt*) from *Q*
- 13 **if** *u* is *t* **then**
- 14 **return** *cnt* - 1
- 15 **for each neighbor** *v* **of** *u* **do**
- 16 **if** *v* not visited **then**
- 17 Mark *v* as visited
- 18 Enqueue (*v*, *cnt* + 1)

19 **return** -1

Question (d)

Analyze the time and space complexity of your algorithm.

Solution:

Time Complexity Analysis:

- Let n be the number of tram routes and m the maximum number of islands in any route.
- Building the mapping from islands to routes requires visiting each island in each route once, taking $O(n \cdot m)$ time.
- Constructing the graph of tram routes: for each island, all routes passing through it are connected pairwise. In the worst case, this requires $O(n^2)$ operations.
- BFS traversal visits each route node once and explores all edges once, taking $O(n + E)$ time, where E is the total number of edges, at most $O(n^2)$.
- Therefore, the overall time complexity is:

$$O(n \cdot m + n^2)$$

Space Complexity Analysis:

- Storing the mapping from islands to routes requires $O(n \cdot m)$ space.
- Storing the adjacency list of tram routes requires $O(E)$ space, at most $O(n^2)$.
- BFS queue and visited array use $O(n)$ space.
- The total space complexity is therefore:

$$O(n \cdot m + n^2)$$